

Informe Final: Análisis de Seguridad y Vulneración de Controles de Acceso en la Plataforma de Reserva de Salas UC (25Live)

Matías Nicolás Fernández Taípe
Curso: Seguridad Computacional
Pontificia Universidad Católica de Chile

24 de junio de 2026

1. Alcance del Proyecto

Este proyecto consistió en un análisis de seguridad ofensiva y auditoría de APIs sobre la plataforma **25Live de CollegeNET**, el sistema de gestión y reserva de espacios físicos utilizado por la Pontificia Universidad Católica de Chile. El alcance del trabajo incluyó la interceptación, el análisis de la estructura de datos de las solicitudes (*requests*) y respuestas (*responses*), y la posterior replicación de peticiones HTTP con manipulación de parámetros de estado e identidad.

El objetivo final ejecutado exitosamente fue vulnerar las restricciones perimetrales impuestas en la interfaz de usuario de la plataforma para forzar la reserva inmediata y confirmada de salas de clases en el campus para una fecha cercana, un privilegio que debería estar estrictamente restringido a los administradores del sistema o unidades académicas autorizadas.

2. Contextualización y Modelo de Amenazas

El control de acceso y la autorización son la base de la seguridad. En la universidad, los espacios físicos son recurso clave y bastante limitado. El problema actual con la UC es que restringe el uso de esta plataforma mediante un flujo de asignación poco transparente. Un estudiante común y corriente tiene bloqueada la opción de pedir fechas cercanas en la interfaz, lo que nos obliga a interactuar con bloques de tiempo absurdamente lejanos, como el año 2028.

Este escenario plantea una relevancia multidimensional en seguridad computacional:

- **Falla de Diseño en Autorización (BOLA / IDOR):** Confiar la seguridad exclusivamente al front-end (restricciones visuales de la interfaz) sin validación de permisos estricta en el servidor.
- **Superficie de Ataque y Denegación de Servicio (DoS) Físico:** Al permitir que un rol sin privilegios apruebe sus propias solicitudes, se abre la puerta a la automatización de scripts que bloqueen masivamente los espacios del campus, paralizando la actividad académica regular.
- **Fuga de Información Semántica:** Exposición indebida de datos sobre la estructura interna corporativa e identidades de los funcionarios encargados del sistema.

3. Metodología y Herramientas

La auditoría se ejecutó siguiendo las siguientes fases técnicas en un entorno controlado:

1. **Reconocimiento y Evasión del Perímetro:** Autenticación legítima en el portal `25live.collegenet.com`. Para evadir el bloqueo del front-end, se generó un flujo inicial completando el formulario para la fecha permitida (2028).
2. **Interceptación de Tráfico:** Monitoreo de la pestaña *Network* de las DevTools del navegador para capturar la transacción JSON generada al presionar “Save”.
3. **Ingeniería Inversa del Payload:** Aislamiento de las variables críticas de la máquina de estados, parámetros temporales (`start_date`, `reservation`) e identificadores de espacio (`space_id`).
4. **Replicación y Envío de Peticiones:** Configuración de las requests utilizando **Apidog**, empleado estrictamente como una interfaz gráfica y cliente HTTP cómodo para estructurar, organizar y enviar de forma limpia las peticiones simuladas. En esta interfaz se inyectaron las cabeceras obligatorias de sesión (`Cookie` con tokens `WSSESSIONID`, `Blaze`, y `_shibsession_`) para evadir respuestas 401 `Unauthorized`.
5. **Elevación de Privilegios:** Manipulación controlada del parámetro `state` del JSON para forzar transiciones de estado no autorizadas.

4. Evidencia Técnica y Hallazgos

Los experimentos de inyección arrojaron el descubrimiento y documentación de las siguientes funciones subyacentes del ecosistema de APIs de CollegeNET:

4.1. Catálogo de Endpoints Documentados

4.1.1. 1. Modificar o Re-agendar un Evento Existente

- **Método HTTP:** PUT
- **URL EndPoint:**

```
https://25live.collegenet.com/25live/data/uccl/run/rose/event.json?
event_id=136061&return_doc=T&caller=pro-EventService.putEvent
```

- **Propósito y Comportamiento:** Permite alterar las propiedades de un evento previamente guardado (nombre, organizaciones, roles o fechas). Exige la actualización interna de bloques JSON como `start_date`, `init_start_dt` y el arreglo de `reservation`. Las salas físicas se solicitan dentro de `space_reservation` mediante su correspondiente ID numérico (`space_id`).

4.1.2. 2. Buscar Salas Libres (Disponibilidad Real)

- **Método HTTP:** GET
- **URL EndPoint:**

```
https://25live.collegenet.com/25live/data/uccl/run/spaces.json?
scope=list&query_id=140&min_capacity=15&max_capacity=20&
can_schedule=T&sort=normal_name&perm_start_dt=YYYY-MM-DDTHH:MM:
SS&perm_end_dt=YYYY-MM-DDTHH:MM:SS&caller=pro-SpaceService.
getSpacesBySearchQuery
```

- **Propósito y Comportamiento:** Cruza la disponibilidad del sistema en tiempo real para retornar únicamente los espacios físicos libres en el bloque horario exacto definido por `perm_start_dt` y `perm_end_dt`, respetando los límites de capacidad e identificando dónde se poseen permisos operativos (`can_schedule=T`).

4.1.3. 3. Listar el Catálogo General de Salas

- **Método HTTP:** GET
- **URL EndPoint:**

```
https://25live.collegenet.com/25live/data/uccl/run/list/listdata.
json?compsubject=location&sort=name&order=asc&page=1&page_size
=25&obj_cache_accl=0&min_capacity=11&caller=pro-ListService.
getData
```

- **Propósito y Comportamiento:** Retorna la lista maestra o catálogo general de los espacios del campus (filtrados por capacidad mínima). Sirve para mapear qué `space_id` corresponde a cada aula oficial, funcionando incluso sin requerir parámetros de autenticación complejos.

4.1.4. 4. Buscar Usuarios / Contactos del Sistema

- **Método HTTP:** GET
- **URL EndPoint:**

```
https://25live.collegenet.com/25live/data/uccl/run/search-criteria/
contact/contacts.json?obj_cache_accl=0&page=0&page_size=100&name
=busqueda&is_r25user=1&caller=pro-SearchCriteriaService.
getContacts
```

- **Propósito y Comportamiento:** Permite realizar búsquedas en la base de datos de la universidad mediante el parámetro `name`. Devuelve el `contact_id` de profesores, alumnos o coordinadores y expone sus correos institucionales, lo cual facilita la asignación de roles de *Requistor* o *Scheduler* en los payloads de explotación.

4.2. El Quiebre del Mecanismo de Autorización

El ataque contra la lógica de negocio se consolidó mediante la alteración secuencial de la máquina de estados del servidor:

- **Fase de Draft (state: 0):** Al enviar la petición inicial para evadir la interfaz gráfica, el sistema generó el código localizador de borrador 2026-AARRNC. El motor de asignación detectó que el rol no poseía privilegios de reserva directa y degradó automáticamente las salas físicas solicitadas (Sala ID 761: 01 102 Economía, Sala ID 762: 01 103 Economía y

Sala ID 983: 46 R204 Campus Oriente) removiéndolas del calendario real y etiquetándolas como simples “preferencias” en espera de aprobación manual.

- **Bypass Temporal:** Al manipular directamente los bloques de fechas del Body de la request hacia una fecha inmediata (v.g., el día posterior), el servidor aceptó el cambio sin validar la restricción perimetral que el front-end imponía de forma artificial.
- **Fuga de Información Crítica Intermedia (state: 1):** Al forzar el estado a *Tentative* (state: 1), la respuesta de la API devolvió la estructura interna de aprobación del flujo, filtrando hacia la cuenta del alumno los nombres completos y correos de las funcionarias administrativas encargadas de revisar el espacio (Valdes Loyola, Sandra P. y Jara Meneses, Lisset).
- **Broken Function Level Authorization (state: 2):** El hallazgo crítico se produjo al modificar el parámetro entero a state: 2. El motor central de validación del servidor de CollegeNET procesó la solicitud asumiendo de manera ciega la legitimidad de la orden administrativa. El estado del evento mutó instantáneamente a “**Confirmed**”, consolidando la reserva en el calendario real del sistema y saltándose por completo las alertas de conflicto y las colas de autorización manual de la universidad.

4.3. Guía de Reproducción Controlada

Para validar los hallazgos descritos de manera segura en un entorno de auditoría institucional y justificar la reproducibilidad exigida, se definen las siguientes instrucciones mínimas haciendo uso de un cliente HTTP:

1. Iniciar sesión de forma legítima con credenciales UC en el portal `25live.collegenet.com/pro/uccl` y abrir la consola de herramientas de desarrollador (*DevTools*).
2. Generar un borrador de evento genérico para capturar en la pestaña *Network* los valores correspondientes a los encabezados obligatorios de autenticación: `WSSESSIONID`, `Blaze` y `_shibsession_`.
3. Utilizar una interfaz de cliente HTTP (como Apidog o cualquier software equivalente de software libre/Postman) únicamente para montar las solicitudes, ingresando las URLs analizadas y mapeando las cookies de sesión obtenidas en la sección de *Headers*.
4. Mutar los valores temporales del cuerpo JSON a fechas del periodo vigente y modificar de forma directa el parámetro de control "state", reemplazando el valor por defecto 0 por el entero de autorización 2.
5. Ejecutar la solicitud a través de la interfaz gráfica del cliente y verificar el retorno exitoso de un código HTTP 200 OK con el flag de confirmación del inventario físico activo.

5. Lecciones Técnicas, Mitigaciones y Ética

5.1. Limitaciones y Decisiones de Diseño

La principal barrera técnica fue el comportamiento del motor de asignación (`accl`), el cual almacenaba temporalmente las solicitudes degradadas dentro de una etiqueta `<reservation_list>` en texto plano/XML dentro de la respuesta corporativa. El aislamiento y análisis de los flujos mediante peticiones manuales confirmó el diagnóstico: el sistema padece de una arquitectura de “confianza ciega” en los payloads enviados, delegando la seguridad de los estados críticos al cliente en lugar de aplicar una validación robusta en el servidor.

5.2. Trabajo Futuro y Mitigaciones Propuestas

1. **Validación Basada en Roles (RBAC) en el Servidor:** El backend debe cruzar de manera mandatoria el `contact_id` de la sesión autenticada con los privilegios asignados antes de procesar cambios en la máquina de estados. Si un rol estudiantil invoca un payload con `state: 1` o `state: 2`, la API debe bloquear la transacción devolviendo un código 403 Forbidden.
2. **Saneamiento de Salidas (Data Masking):** Modificar las respuestas del servicio `EventService` para ocultar datos de flujos de auditoría interna y nombres de administradores a usuarios no privilegiados.
3. **Principio de Privilegio Mínimo en Métodos HTTP:** Restringir la capacidad nativa de las cuentas de estudiantes para emitir solicitudes de tipo PUT hacia los endpoints críticos de CollegeNET.

5.3. Consideraciones Éticas

En estricta observancia de los lineamientos del curso y la ética profesional, todas las actividades de prueba se llevaron a cabo utilizando credenciales propias de manera controlada. No se provocó la interrupción de los servicios ni se afectaron reservas reales de otros miembros de la comunidad. La validación del fallo se limitó a comprobar la persistencia técnica del estado de confirmación en la base de datos sin incurrir en acaparamiento ni denegación del espacio físico.